



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Video coding principles

Marco Cagnazzo

Multimedia Communications



Outline

The hybrid coder

Motion compensation

The Group of Pictures (GOP)

Operational video coding

MPEG standards

MPEG-1

MPEG-2

Scalability



Outline

The hybrid coder

Motion compensation

The Group of Pictures (GOP)

Operational video coding

MPEG standards



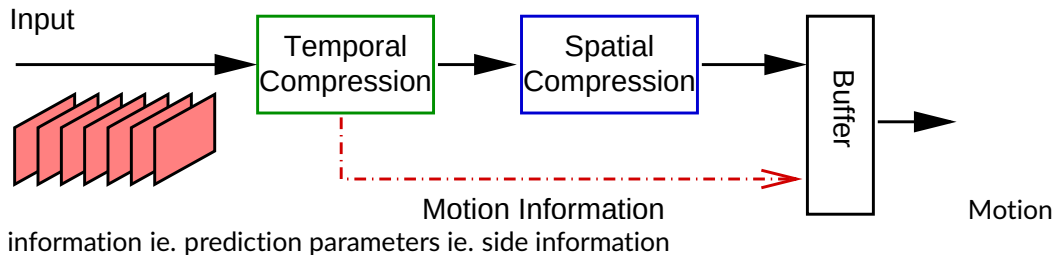
Video compression principles

- ▶ Spatial redundancy
 - ▶ Images are made up of homogeneous regions
- ▶ Time redundancy
 - ▶ Successive images in a video are similar each to the other
- ▶ A video compression algorithm must exploit both kind of redundancy
- ▶ While spatial redundancy is effectively dealt with by linear transform, *prediction* is the most effective tool for removing temporal redundancy
- ▶ Temporal prediction implies computing a predictor and sending the predictor parameters along with the compressed prediction error to the decoder
- ▶ If the prediction error is *sparser* than the signal, the compression is more effective



Video compression principles

General scheme of a video coder





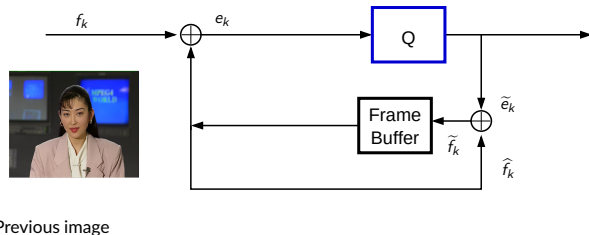
Prediction in video coding: DPCM

- ▶ Successive images are very similar
- ▶ Prediction:
 $\hat{f}_{n,m,k} = \tilde{f}_{n,m,k-1}$
- ▶ No prediction parameter to send
- ▶ What if images are different?

Current image



Error





Block-matching motion estimation

- ▶ For each $B_k^{(p)}$, we look for the most similar block $B_h^{(p+v)}$ in the reference image
- ▶ When we find a “match” we consider that it is the same object, which has been displaced by a vector v
- ▶ The most similar block is the one that minimizes a “distance”, possibly with a regularization term

$$J(v) = d(B_k^{(p)}, B_h^{(p+v)}) + \lambda_{ME} r(v)$$

also referred to as a *cost function*

- ▶ The estimated vector for the block $B_k^{(p)}$ is

$$v^*(p) = \arg \min_v J(v)$$



Performance criteria of motion estimation

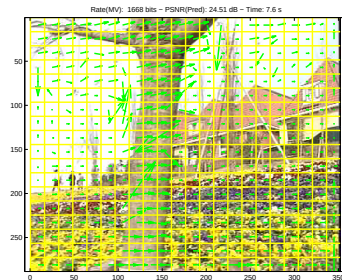
- ▶ Quality of the prediction: MSE between current block and predictor
- ▶ Coding cost: number of bits needed to encode $\mathbf{v}^*$, e.g. with an Exp-Golomb code
- ▶ Computational cost: ME time



Motion estimation

Example

T





- ▶ A prediction $\hat{I}_k$ of image I_k can be build via *motion compensation*
- ▶ The block centered in $\mathbf{p}$ of $\hat{I}_k$ is just a copy of a specific block in the reference image I_h
- ▶ Which block? The one pointed by $\mathbf{v}^*(\mathbf{p})$

$$\hat{l}_k(\mathbf{p}) = l_h(\mathbf{p} + \mathbf{v}^*(\mathbf{p}))$$

- ▶ This prediction is referred to as *motion compensated image*

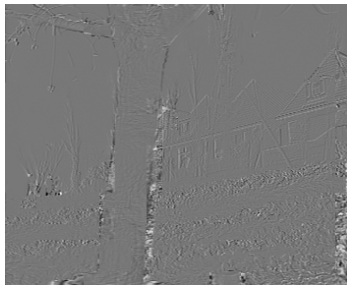


Motion compensation

Motion-compensated image



MC error



Motion-compensated image

MC error $I_k(\mathbf{p}) - \hat{I}_k(\mathbf{p})$



Motion-compensated video coding

- ▶ Video codecs work **block by block**
- ▶ Very often, the motion-compensated block is very similar to the actual block
 - ▶ In this case, it is more effective to encode the prediction error than the current block
 - ▶ Why? Because a *sparser signal* is better compressed with JPEG-like methods
- ▶ But sometimes, the prediction is not effective: new objects, disocclusions
 - ▶ In this case the encoder will encode the actual block and not the prediction error
- ▶ The encoder must signal whether it encodes the current block (*Intra coding*) or the temporal prediction error (*Inter coding*)



Summary of ME/MC: encoder

For each block $B_k^{(p)}$ of the image k :

- ▶ We perform a ME Test with a reference h (e.g. $h = k - 1$):

$$J(\mathbf{v}) = d(B_k^{(p)}, B_h^{(p+\mathbf{v})}) + \lambda_{\text{ME}} r(\mathbf{v})$$

- ▶ The estimated vector corresponds to the best match:

$$\mathbf{v}^* = \arg \min_{\mathbf{v}} J(\mathbf{v})$$

- ▶ The encoder transmits:

- ▶ A signalization bit (Intra: no temporal prediction/Inter: temporal prediction)
- ▶ If Intra, it encodes $B_k^{(p)}$ with a JPEG-like method
- ▶ If Inter, it encodes $\mathbf{v}^*$ and the prediction error $E(\mathbf{p}) = B_k^{(p)} - B_h^{(p+\mathbf{v})}$ with a JPEG-like method



Summary of ME/MC: decoder

For each block $B_k^{(p)}$ of the image k , the decoder reads the signalization bit:

- ▶ If *Intra* it decodes the following bits with a JPEG-like decoder and copies the result in the block $B_k^{(p)}$ of the decoded image
- ▶ If *Inter*
 - ▶ It decodes the motion vector $\mathbf{v}^*$
 - ▶ It decodes the prediction error **block** $E(\mathbf{p})$
 - ▶ The block $B_k^{(p)}$ is built as: $E(\mathbf{p}) + B_h^{(\mathbf{p} + \mathbf{v}^*)}$



Parameters of motion estimation

- ▶ Block size and shape
- ▶ Cost function d
- ▶ Search area
- ▶ Search strategy
- ▶ Motion model



Parameters of motion estimation

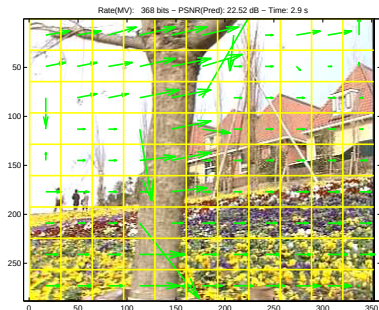
Block size and shape

- ▶ Smaller blocks imply better prediction quality, higher coding cost and higher computational cost
- ▶ Variable block size and shape is very effective in order to use a locally adaptive prediction structure, but also increases the computational cost because the encoder will re-run the ME changing the shape and the size of the blocks

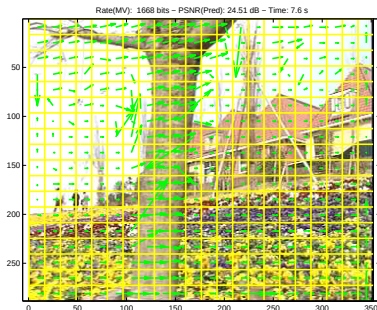


Parameters of motion estimation

Block size and shape



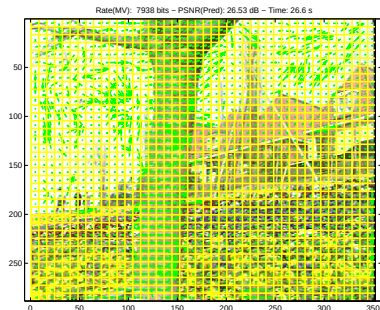
Block size 32×32



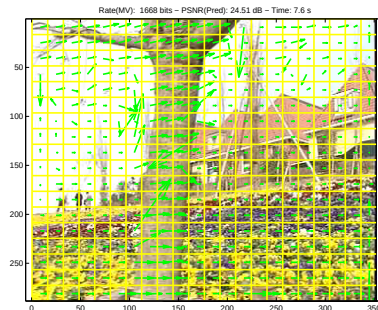
Block size 16×16

Parameters of motion estimation

Block size and shape



Block size 8×8



Block size 16×16



Motion Estimation with Variable Block Size

Input: Frame F , Reference frame R , Initial block size B , λ

Output: Motion vectors and partitioning

for each block b of size B in F do

 Calculate $J(v) = D + \lambda R$ for b ;

 Divide b into four sub-blocks;

for each sub-block s_i do

 | Calculate $J_i(v) = D_i + \lambda R_i$ for s_i ;

end

$J_{sub} = \sum_{i=1}^4 J_i(v)$;

if $J_{sub} < J(v)$ then

 | Choose sub-block partitioning;

 | Recursively apply algorithm to each sub-block;

else

 | Keep original block b ;

 | Store motion vector for b ;

end



Motion Estimation with Variable Block Size (cont.)

Where:

- ▶ D : Distortion measure (e.g., SAD, SSD)
- ▶ R : Rate measure (bits to encode motion vector)
- ▶ λ : Lagrange multiplier for rate-distortion optimization
- ▶ v : Motion vector

Note:

- ▶ The algorithm is applied recursively to sub-blocks
- ▶ Termination condition: minimum block size or $J(v)$ no longer decreases
- ▶ The process balances between motion estimation accuracy and coding efficiency



Parameters of motion estimation

Cost function

Several choices are possible for $d(\cdot, \cdot)$:

- ▶ SAD (Sum of Absolute Differences)

$$d(B_1, B_2) = \sum_{n,m} |B_1(n, m) - B_2(n, m)|$$

- ▶ SSD (Sum of Squared Differences)

$$d(B_1, B_2) = \sum_{n,m} [B_1(n, m) - B_2(n, m)]^2$$

SSD has the smallest MSE, but it is more complex and requires more bits (irregular vectors)



Parameters of motion estimation

Cost function

Regularized cost function:

- ▶ Instead of minimizing $d(\mathbf{v})$, one can minimize a regularized cost function J :

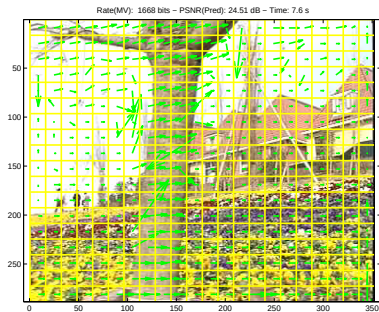
$$J(\mathbf{v}) = d(\mathbf{v}) + \lambda_{ME} r(\mathbf{v})$$

- ▶ $r(\mathbf{v})$ can be the coding cost (number of bits) of vector $\mathbf{v}$
- ▶ Adding the regularization means that we can select a vector that does not achieve the best MSE, but costs less bit to be encoded
- ▶ The trade-off is driven by λ_{ME}
 - ▶ Small λ_{ME} means that the coding cost is not important and $\mathbf{v}$ must only minimize the distance d
 - ▶ Large λ_{ME} means that minimizing the coding cost is what matters, even at cost of a large "distortion" $d(\mathbf{v})$

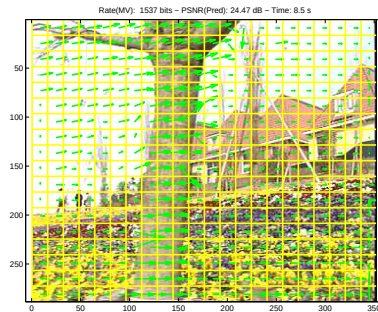


Parameters of motion estimation

Cost function



Non-regularized MVF

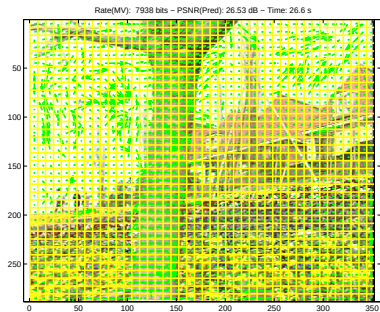


Regularized MVF

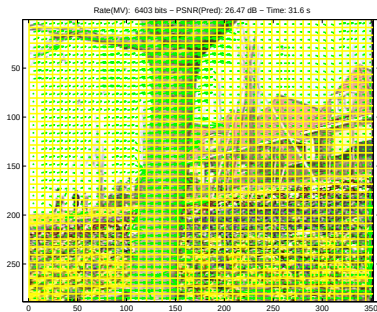


Parameters of motion estimation

Cost function



Non-regularized MVF



Regularized MVF



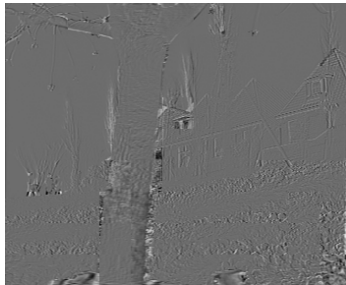
Parameters of motion estimation

Cost function

Motion-compensated image



MC error



Regularized MVF, motion-compensated image

Regularized MVF, compensation error



Parameters of motion estimation

Search radius and strategy

- ▶ The most natural search strategy for ME is the *full search*:
- ▶ In FS all the vectors with coordinates from $-n$ to n are tested
- ▶ This makes a total of $(2n + 1)^2$ computations of the cost function
- ▶ n is referred to as *search radius*
- ▶ Increasing n improves the prediction quality but also increases both the coding cost and the computation time
- ▶ One typically would use the smallest radius that allows to estimate all the displacements



Parameters of motion estimation

Search radius and strategy

- ▶ In FS all the vectors with coordinates from $-n$ to n are tested
- ▶ This guarantees that the least-cost vector within the search area will be found
- ▶ One could give up testing all the $(2n + 1)^2$ vectors of the search area
- ▶ Only a smartly defined subset of vectors is tested: *fast ME*
- ▶ Fast ME typically achieve quasi-optimal prediction quality at a fraction of the computation time required by FS
- ▶ The coding cost is not much impacted by fast ME



Parameters of motion estimation

Search strategy

Some of the Fast ME strategies are:

- ▶ *Cross search* method: First horizontal vectors are tested; then the vertical component is found; total, $2n$ vectors
- ▶ *Log search* method: Nine positions $\{0, \pm(2^m - 1)\}^2$ are tested; the search window is then centered on the best position and the search step is halved to $2^{m-1} - 1$ pixels. The number of tests is $\approx 8 \log_2 n$
- ▶ *Diamond search* method: Eight directions are tested, but the step is reduced only when the center position has been chosen



Block-matching motion estimation

Motion model

- ▶ We look for the rigid displacement $\mathbf{v}$ that makes the prediction block most similar to the current one
- ▶ We could use more complex motion models, eg. including rotation, zoom, perspective deformation
- ▶ Mathematically speaking, an *affine motion model* could be applied, and its parameters could be optimized
- ▶ The affine motion models largely increases the complexity and the coding cost of motion, but can improve the prediction quality
- ▶ It is only used in the most recent video compression standard H.266/VVC



Motion estimation

Summary

- ▶ Goal: sparsify the signal
- ▶ How: by providing a prediction of the current block
- ▶ It is very effective and used virtually all video encoders
- ▶ Performance indicators: quality - coding cost - complexity
- ▶ Design choices:
 - ▶ Cost function (SAD, SSD, regularization, . . .)
 - ▶ Shape and size of blocks
 - ▶ Search radius and strategy
 - ▶ Motion model (translational/affine)



Outline

The hybrid coder

The Group of Pictures (GOP)

Operational video coding

MPEG standards



The Frame types and the Group of Pictures

In all video compression methods, frames are divided into

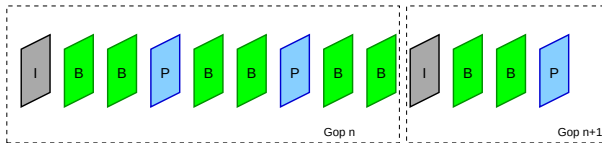
- ▶ I Frames (Intra coded)
- ▶ P Frames (Predictive)
- ▶ B Frames B (Bi-directional or bi-predictive)

In older standards, I and P Frames are referred to as Anchor Frames (AF).
We will first describe the frame organization based on AF.



Group of Pictures

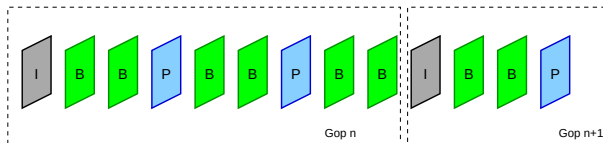
- ▶ The frames are organized into a periodical structure called **GOP** (Group of Pictures)
- ▶ The GOP defines how each frame can be predicted based on other frames
- ▶ The concept of GOP is central in all video standards
- ▶ We start by describing the first GOP architectures (MPEG-1 and MPEG-2):
 - ▶ The first frame of the GOP is **always an Intra frame**
 - ▶ Between 2 consecutive AF one finds a fixed number of B frames (possibly zeros)
 - ▶ Thus GOP structure is completely defined by:
 - ▶ the interval between I frames (GOP period)
 - ▶ the number of B frames between successive AFs





I Frames

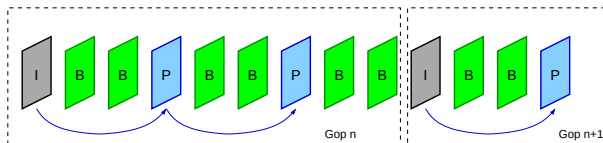
- ▶ Encoded independently from others
- ▶ JPEG-like coding
- ▶ Low complexity, low coding rate
- ▶ Used for:
 - ▶ Random access: they can be decoded independently from others
 - ▶ Fast forward: Decoding only I-frames allows to have an accelerated video playback
 - ▶ Error robustness: They stop error propagation
- ▶ All the blocks of an Intra frame are Intra coded





P Frames

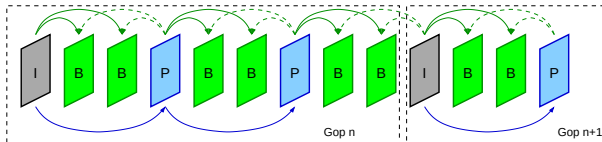
- ▶ Each P frame can be predicted from the the previous AF
- ▶ The prediction decision is taken at block level: each block of a P frame can be either Inter coded or Intra coded
- ▶ P frames have high complexity because of
- ▶ But also a much higher compression ratio than I for the same quality





B Frames

- ▶ They can be predicted from both previous and next AF
- ▶ For each block the encoder can decide if use Intra coding, forward prediction (predict from the past), backward prediction (predict from the future) or bi-directional prediction (use the average of past and future as prediction)
- ▶ Very high complexity (double ME)
- ▶ Very high compression ratio for the same quality

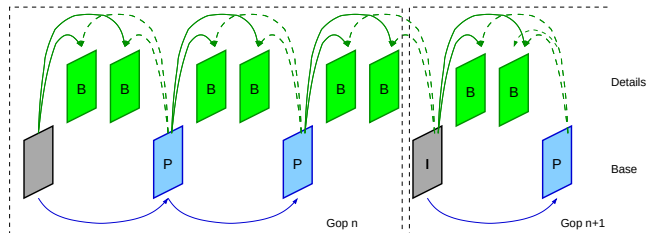




Frame coding order

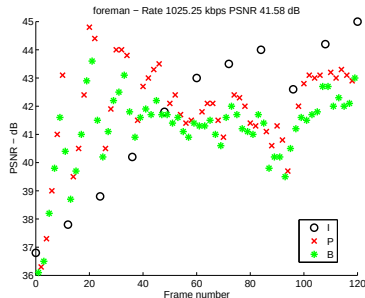
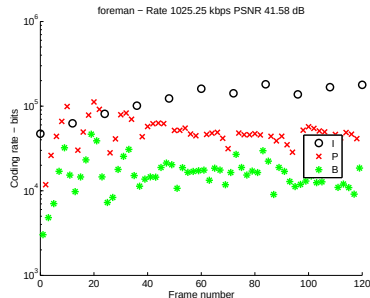
$I \rightarrow AF \rightarrow \text{Frames } B \rightarrow AF \rightarrow \text{Frames } B \dots$

Delay?





Rate and PSNR of different frame types



After some adjustment, we observe that I frames are approximately 3 to 5 times larger than P and 10 to 20 times larger than B

PSNR is more variable and depends on content, but the quality of I frames is typically forced to be high because they are used for predicting the whole GOP



Outline

The hybrid coder

The Group of Pictures (GOP)

Operational video coding

MPEG standards



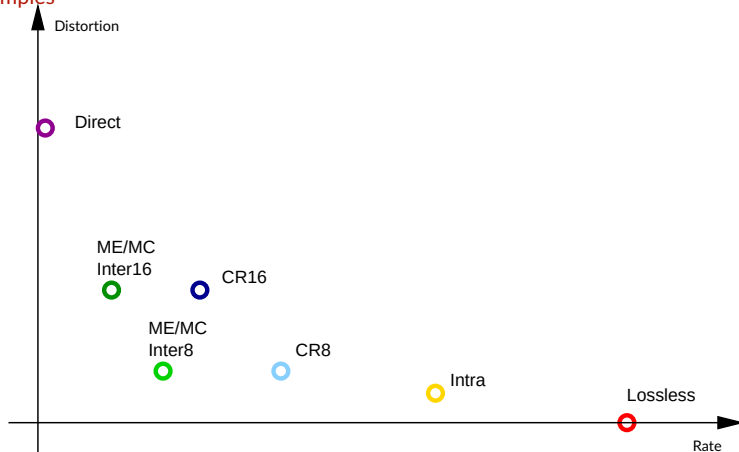
The hybrid video encoder

- ▶ All practical video codecs work on the basis of blocks of pixels
- ▶ These blocks are referred to as **macrobloks** (older standards until H.264) or **coding units** (from H.265 on)
- ▶ Each MB (or CU) can be coded using one among several *coding modes*
- ▶ The set of available coding modes depends on the frame type
 - Intra:** No temporal prediction, transform-based coding; available for all frames
 - Inter:** ME/MC-based temporal prediction, transform coding; available only non-Intra frames
 - Direct:** Motion vector inferred from neighbors; no residual coding; available only non-Intra frames
 - Lossless** available on all frames
- ▶ Each mode has different rate and distortion, how to choose? This is the **mode selection problem**



The hybrid video encoder

Coding performance examples





Coding mode selection

- ▶ A video sequence is composed by K blocks of pixels.
- ▶ Goal of coding mode selection: for each block k , find the coding mode i_k such that D is minimized for a given total rate R :

$$D = \sum_{k=1}^K D_k(i_k, Q)$$

$$R = \sum_{k=1}^K R_k(i_k, Q)$$

- ▶ The quantization step Q is given
- ▶ The set of modes $\mathbf{i} = \{i_k\}_{k=1}^K$ must be chosen such that we minimize:

$$J(\mathbf{i}, Q, \lambda) = \sum_{k=1}^K D_k(i_k, Q) + \lambda \sum_{k=1}^K R_k(i_k, Q)$$



Coding mode selection

- ▶ Joint minimization over i is way too complex
- ▶ A sub-optimal minimization is preferable
- ▶ For a block k , we choose the mode such that we minimize:

$$J_k(i_k, Q, \lambda) = D_k(i_k, Q) + \lambda R_k(i_k, Q)$$

- ▶ That is, we minimize *separately* each term of the sum that gives J
- ▶ The selected mode depends on Q and λ



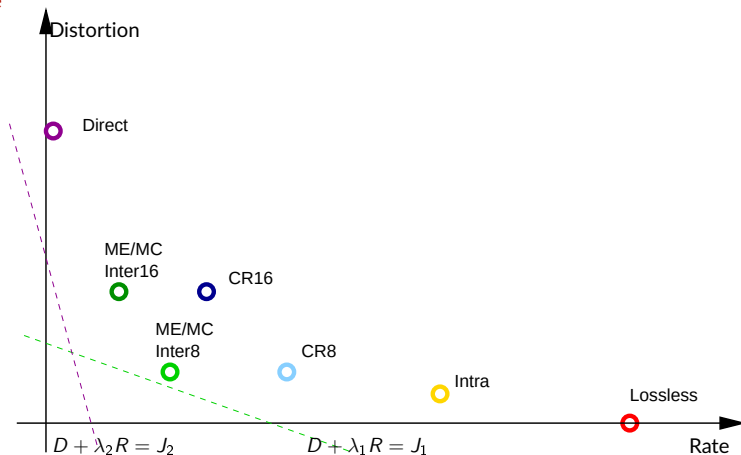
Coding mode selection

- ▶ The quantization step Q is considered as an *input*
- ▶ For each Q (rate) there exists an optimal λ value, which is determined empirically for each different video codec. Examples:
 - ▶ MPEG-2 : $\lambda = aQ^2 + b$
 - ▶ H.264 : $\lambda = c2^{dQ+e}$
- ▶ With a fixed λ , minimizing J_k amounts to find a line in the RD plane



The hybrid video encoder

Example of performance



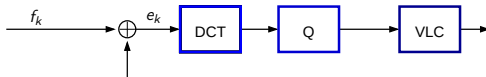


The hybrid video encoder

- ▶ Now we have all the tools for building an hybrid video encoder and decoder
- ▶ Each image is split into blocks
- ▶ For each block, we can compute a prediction (possibly with a null predictor)
- ▶ The prediction error is JPEG-like compressed (DCT+Quantization)
- ▶ The block is then decoded, because it might serve for further predictions
- ▶ The temporal prediction is performed via ME/MC
- ▶ Motion vectors and quantized coefficients are encoded with a VLC

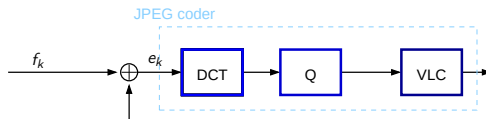


The hybrid video encoder



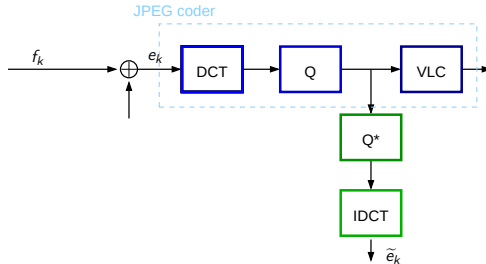


The hybrid video encoder



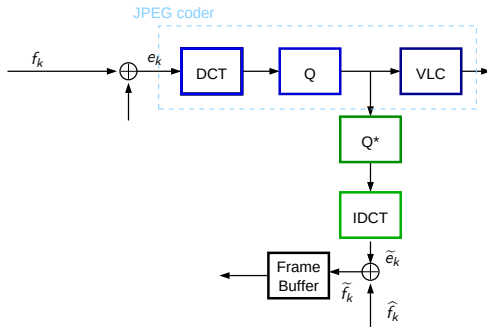


The hybrid video encoder



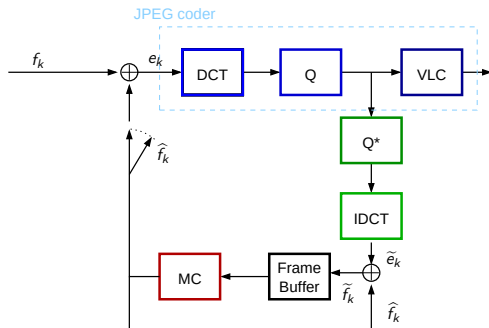


The hybrid video encoder



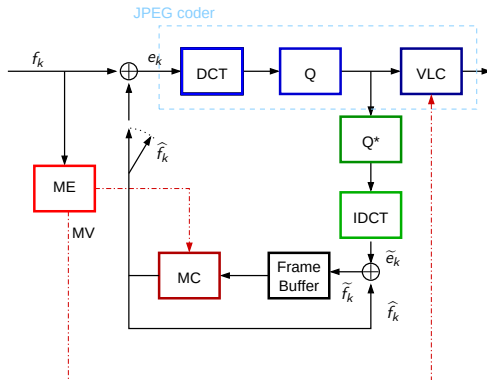


The hybrid video encoder



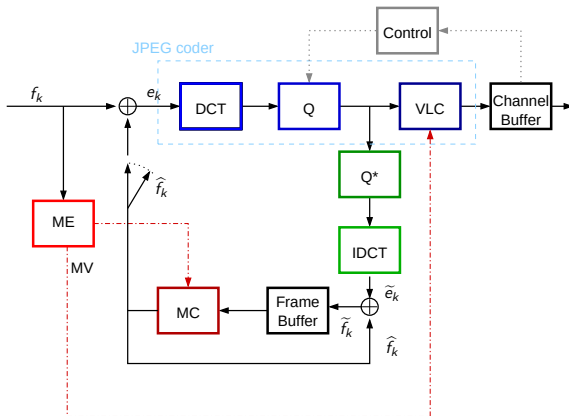


The hybrid video encoder





The hybrid video encoder



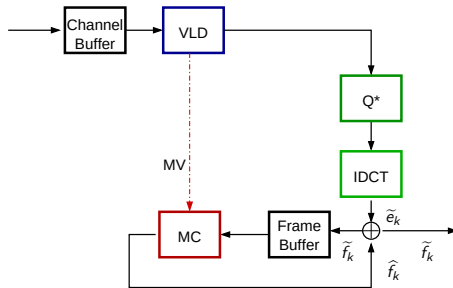


The hybrid video decoder

- ▶ The decoder is a subset of the encoder
- ▶ The current block is JPEG-decoded
- ▶ If needed, the prediction is built by using decoded motion vectors
- ▶ The decoded image is stored because it can be used as further reference



The hybrid video decoder



Asymmetrical scheme!



Hybrid video coding: wrap-up

- ▶ Each frame in the sequence is labeled as I, P, or B according to the GOP structure
- ▶ The encoder works on a block-by-block basis
- ▶ For each block the encoder can choose one *coding mode* within a set that depends upon the frame type
 - ▶ For example, no motion-compensated mode can be used in I frames
- ▶ The *coding mode decision* strategy is not standardized
- ▶ The **mode syntax** is standardized
 - ▶ This means that an encoder can take a suboptimal mode decision and the encoded stream will still be decodable
 - ▶ It can make sense if the goal is to reduce the computation time
- ▶ The optimal RD decision (dubbed as RDO) is the one minimizing $J = D + \lambda R$

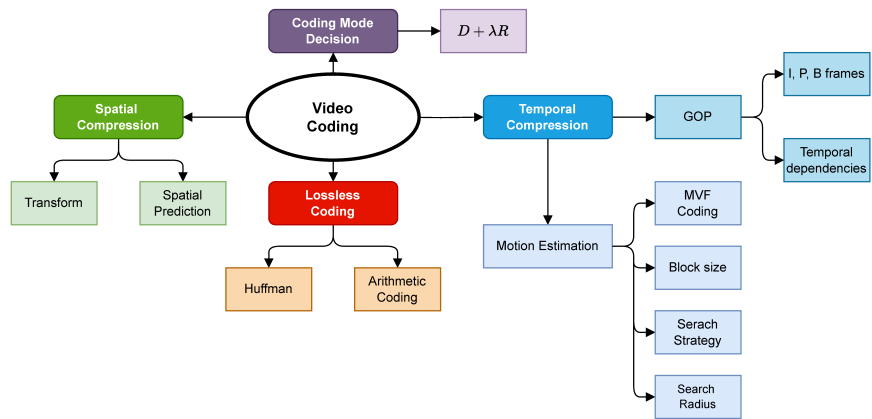


Hybrid video coding: wrap-up

- ▶ In **INTRA coding mode**, only information from the current frame can be used to encoded the block
 - ▶ Spatial coding tools can be used: linear transform (DCT) and possibly spatial prediction
- ▶ In **INTER coding mode**, information from decoded frames *other than the current one* in the DFB can be used:
 - ▶ Temporal prediction for P frames
 - ▶ Bi-prediction from B frames (information from two frames in the DFB)
 - ▶ ME parameters have a big impact on the rate-distortion-complexity trade-off
 - ▶ **Motion vector must be encoded losslessly:**
 - ▶ Huffman, Arithmetic or Exp-Golomb coding
 - ▶ Since motion vector fields typically have spatial coherence, predictive coding is often used
- ▶ The channel buffer allows to adapt coding rate to the rate of the output channel or of the storage medium
 - ▶ When the channel buffer size is above a high threshold, the quantization parameter is increased (more compression)
 - ▶ When the channel buffes size is below a low threshold, the quantization parameters is reduces (less compression)



Hybrid video coding: concept map





Outline

The hybrid coder

The Group of Pictures (GOP)

Operational video coding

MPEG standards

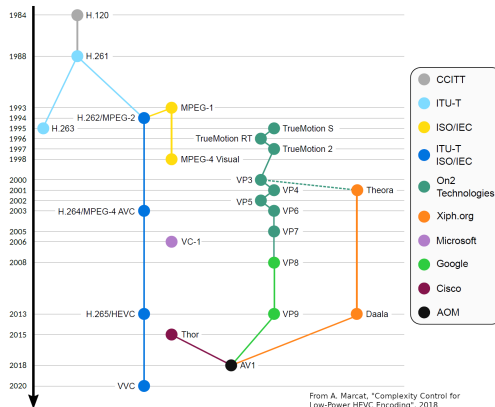
MPEG-1

MPEG-2

Scalability



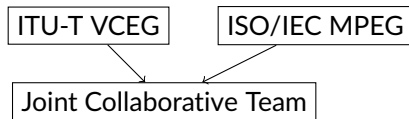
Video standards





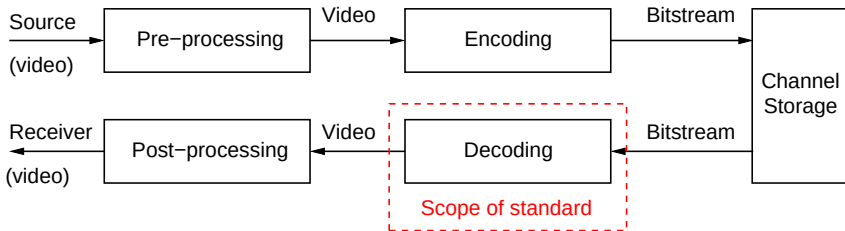
Key Players in Video Codec Standardization

Organization	Role and Contributions
ITU-T VCEG	H.26x series (e.g., H.264/AVC, H.265/HEVC)
ISO/IEC MPEG	MPEG standards (e.g., MPEG-2, MPEG-4)
Joint Collaborative Team	JCT-VC (H.265/HEVC), JCT-VVC (H.266/VVC)
Alliance for Open Media	AV1 codec
DVB Project	Digital video broadcasting standards





Standard Scope



- ▶ The standard only defines the bitstream syntax and the decoder behavior
- ▶ Goal: interoperability, competition



MPEG-1 standard

- ▶ Developed in 1988-1992
- ▶ Parts
 1. Systems
 2. Video
 3. Audio
 4. Conformance test
 5. Software simulation



MPEG-1 standard

Part 2 (Audio)

- ▶ MPEG-1 Part 2 is the audio section of the MPEG-1 standard
- ▶ Officially known as ISO/IEC 11172-3
- ▶ Defines three audio coding layers:
 - ▶ Layer I
 - ▶ Layer II
 - ▶ **Layer III (MP3)**
- ▶ MP3 (MPEG-1 Audio Layer III) is the most well-known format
- ▶ **Revolutionized digital audio distribution**
- ▶ Lossy compression, **perceptually lossless**
- ▶ Typical bitrates: 128-320 kbps
- ▶ Compression ratio up to 12:1
- ▶ Widely supported across devices
- ▶ Still widely used (20% market share), declining



MPEG-1 standard

Part 3 (Video)

- ▶ Hybrid coder with ME/MC
- ▶ Input: max 720×576 pixel @ 30 fps
- ▶ Rate ≤ 1.86 Mbps (VHS quality)
- ▶ Asymmetric applications: VoD, video CD, videogames

Features

- ▶ Image types
- ▶ Sub-pixel (fractional) ME/MC

It is an obsolete technology but has a huge historical importance as the first international standard for audio and video coding. Most of the ideas of MPEG-1 are still used in modern codecs (hybrid architecture, frame types, fractional ME/MC)



MPEG-2 standard

- ▶ Developed in 1990-1994
- ▶ Technically almost identical to MPEG-1, but with extended input parameter space
- ▶ Parts
 1. Systems
 2. Video
 3. Audio
 4. Conformance test
 5. Software simulation



MPEG-2 standard

- ▶ Hybrid coder
- ▶ Profiles and levels
- ▶ Rate ≤ 15 Mbps ("Main" profile)
- ▶ Interlaced video support
- ▶ Scalability support



MPEG-2 standard

Profiles and levels

Profiles:

- ▶ Define subsets of the bitstream syntax
- ▶ Determine algorithmic features

Levels:

- ▶ Specify the range of values for parameters
- ▶ Determine computational complexity and memory requirements

- ▶ Each profile can be implemented at different levels
- ▶ Higher profiles/levels = More features/quality, but more complex
- ▶ Decoders must support all features up to specified profile/level



MPEG-2 standard

Profiles and levels

Level	width [pixel]	height [pixel]	frame rate [frame/s]	bit rate [Mbps]
Low	352	288	30	4
Main	720	576	30	15
High-1440	1440	1152	60	60
High	1920	1152	60	80

Profile	Feature
Simple	No scalability; interlaced video; no B-frames
Main	Simple + B-frames
SNR scalable	Main + Two or three quality scalability levels
Spatial scalable	SNR + Two or three resolution scalability levels
High	Space + Oversampled chroma



MPEG-2 standard

Profiles and levels

Level	Profile				
	Simple	Main	SNR scalable	Spatial scalable	High
Low		•	•		
Main	•	•	•		•
High-1440		•		•	•
High		•			•



Current Usage of MPEG-2

Main Applications of MPEG-2 Today:

- ▶ Standard Definition TV in several countries
- ▶ Some HD broadcasts in developing regions
- ▶ Still the standard format for DVDs
- ▶ Satellite and Cable TV Services
- ▶ Still used due to legacy systems
- ▶ Gradually being replaced by newer standards (H.264, H.265)
- ▶ Remains relevant in areas
- ▶ Continued use in professional broadcast environments



Video content distribution

- ▶ Video content distribution model: one source (server), many destinations (users)
- ▶ Users differ among each other:
 - ▶ Different network paths to the server and thus different rates, delays
 - ▶ Time-varying channel conditions, in particular for wireless channels
 - ▶ Different available memory and computational power
 - ▶ Different screens (from smartphones to 4K TV sets)
- ▶ Yet they all ask for the “same” video content
- ▶ How to decide the coding rate?
- ▶ This mostly depend on the numer of bits per second that can be transferred from the server to the user k : let us refer to this quantity as $C(k)$ (channel capacity or link capacity)



The coding rate constraint

- ▶ The coding rate R_C is the number of bits per second needed to represent the video content after compression
- ▶ If $C(k) < R_C$, the connection “is not fast enough”
- ▶ Example.
 - ▶ Consider a video encoded at 10Mbps and transmitted over a link with $C(k) = 5$ Mbps
 - ▶ Now imagine that one second of video is transmitted as a GOP
 - ▶ It amounts to 10 Mbits...
 - ▶ ... but it takes 2 seconds to be received
 - ▶ The client will play one second of video and then wait another second to start over playing
 - ▶ In this case, the quality of experience is catastrophic (cliff effect)
- ▶ **Thus we need to impose the constraint $R_C \leq C(k)$**



Cliff and levelling effects

- ▶ Since since each user k sees a different capacity $C(k)$ for the connection to the server, to impose the constraint one should set $R_C = \min_k C(k)$
- ▶ However, users with capacity larger than R_C will suffer from levelling, i.e., they could have a better quality but they are not allowed to fully exploit their channel
- ▶ In conclusion, any choice of R_C will harm the QoE of some users because the same coding rate is imposed to everyone
- ▶ We would like to have an **adaptive coding rate**



Simulcast

- ▶ A first, straightforward solution consists in **simulcast**
- ▶ With simulcast, the original video content is encoded at N different bit-rates $R_1 < R_2 < \dots < R_N$
- ▶ The server sends to user k with the maximum rate less than its channel capacity
- ▶ The different coding rates can be obtained by adjusting
 - ▶ The quantization step
 - ▶ The video resolution
 - ▶ The frame rate
- ▶ A suitable protocol is used between users and server, such that each user is dynamically provided with the best version of the video compatibly with the rate constraint



Simulcast: pros and cons

► Pros:

- Simple approach
- Does not require modifications to the codecs
- Can be implemented at the client side: the client estimate the channel capacity (e.g., by measuring the RTT of a packet) and asks for the suitable encoded video
- Solves both per-user and temporal channel variations

► Cons:

- The server must store many version of the videos
- The network resources are suboptimally used, since many redundant versions of the same content travel on it
- It is difficult to adapt this approach to live or real-time services, because in this case we should run N parallel encoders at the server side



Scalability

An alternative solution consists in *scalable coding*:

Encode once, decode many!

- ▶ The video sequence is compressed in a **single** bitstream that can be decoded at N different rates
- ▶ The bitstream is made up of:
 - ▶ One *base* layer
 - ▶ $N - 1$ *enhancement* layers
- ▶ Each layer can be **sent** independently from the others

Example: the base layer is the video compressed with a given quantization step; the enhancement layer consists in compressing the details lost at base layer (i.e., the difference between the base layer and the original video) with a smaller quantization step



The base and the enhancement layers

- ▶ The **base layer** can be decoded alone and provides a minimum-quality service
- ▶ The bit-rate R_B of the base layer is typically rather small in such a way that $R_B \leq \min_k \{C(k)\}$
- ▶ The low bit-rate of the base layer can be obtained by adjusting one or more of the following parameters
 - ▶ Quantization step (quality or SNR scalability)
 - ▶ Resolution (space scalability)
 - ▶ Frame rate (time scalability)
- ▶ The **enhancement layers** allow to improve quality or resolution or frame-rate ...
- ▶ ... but they are useless without the base layer



Advantages of scalability

1. The video is encoded only once, thus saving storage space on the server
2. Also, one encoding step means a better fit with live or real-time applications
3. Scalability allows for efficient exploitation of network resources (see next slides)
4. A scalable video can adapt at **transmission time** to the network conditions
5. This allows not only per-use adaptation, but also dynamic (i.e.m, time-varying) adaptation to each single user

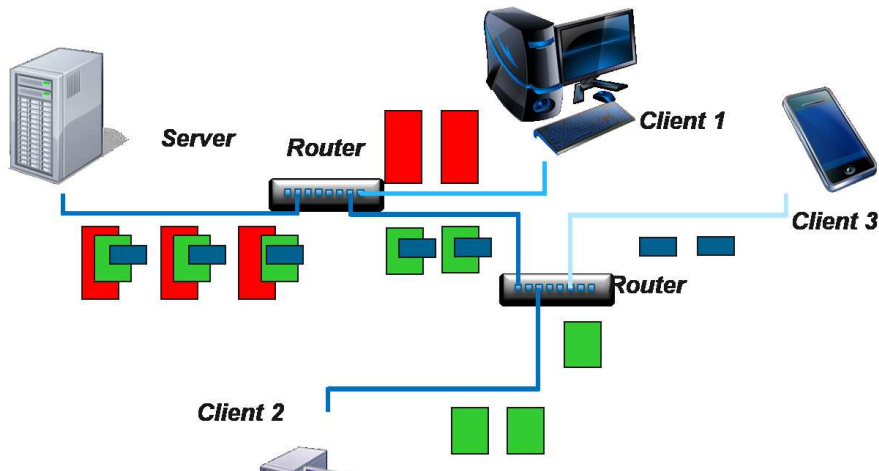


Disadvantages of scalability

1. Scalability requires a deep modification of the hybrid video codec
2. For the single user, the quality achievable with the same rate is typically somewhat less than the simulcast case
3. Fully exploiting the network requires more advanced routers (modifications at the level 3 network protocols)

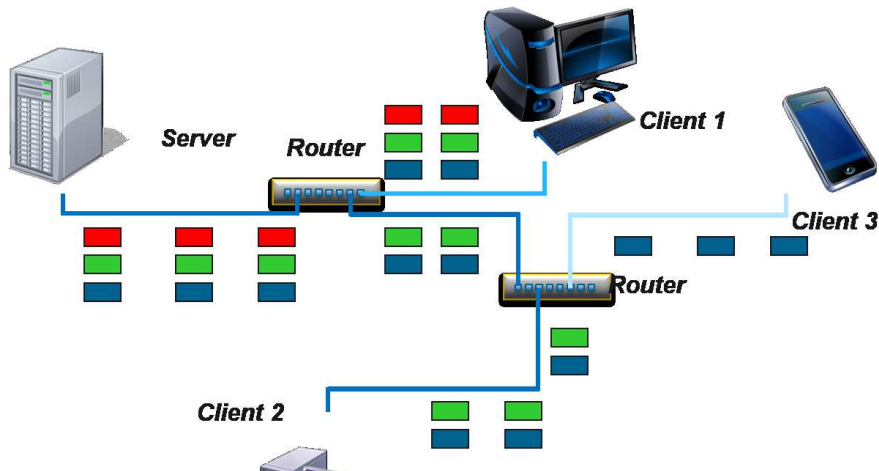


Video distribution without scalability



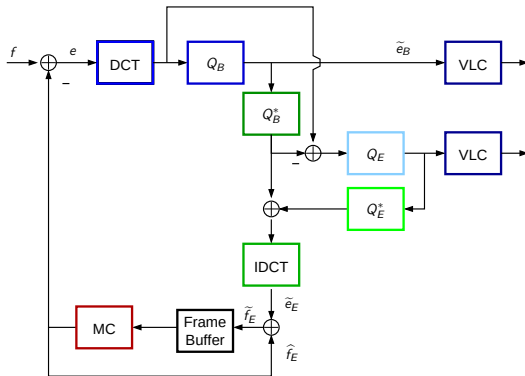


Video distribution with scalability





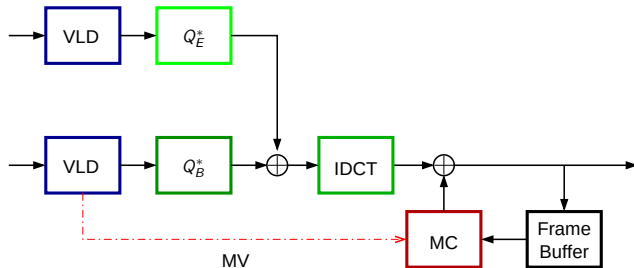
SNR scalability in MPEG-2: encoder



- ▶ DCT coefficients refinement
- ▶ The residual is computed using the the enhancement layer → *drift* of the base layer
- ▶ No drift control

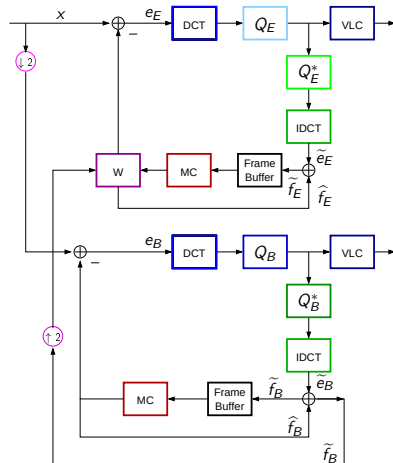


SNR scalability in MPEG-2: decoder





Resolution scal. in MPEG-2: encoder



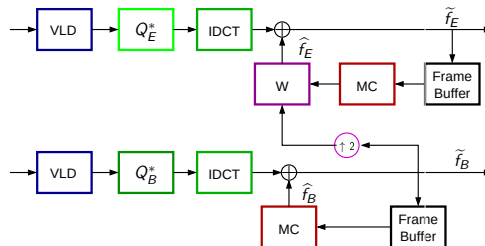


Resolution scal. in MPEG-2: encoder

- ▶ Double loop: no drift
- ▶ Input video is filtered and subsampled
- ▶ Enhanced level prediction is a weighted sum of:
 - ▶ The interpolated base-layer image
 - ▶ ME/MC prediction
- ▶ The weight is changed per each coding block, and its value is encoded in the bitstream



Resolution scalability in MPEG-2: decoder





Time scalability in MPEG-2

